Complexidade de algoritmos

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Algoritmos e Estruturas de Dados II
Prof. Fabrício Sérgio de Paula

Tópicos

- Complexidade de algoritmos
- Notação assintótica
- Exercícios

Complexidade de algoritmos

- Tempo de execução: característica importante de um algoritmo
  - Possibilita prever recursos necessários para execução
- Cálculo do tempo de execução
  1. Método empírico: mede-se o tempo durante execução
     - Diversas entradas distintas
     - Tempo depende de software/hardware usado
  2. Método analítico: modelo matemático
     - Independe de hardware/software
     - Nem sempre é simples

Complexidade de algoritmos

- Método analítico
  - Tempo de execução medido em função do tamanho da entrada
    - Exemplos:
      - Ordenação: quantidade de elementos no vetor
      - Multiplicação de dois números grandes: quantidade de bits
  - Considera <u>operação dominante</u>: operações básicas com maior frequência de execução
    - Exemplos:
      - Ordenação: comparação de elementos
      - Multiplicação de matrizes: multiplicações realizadas

Complexidade de algoritmos

- Exemplo 1: Inversão de uma sequência

```
algoritmo 1.1: Inversão de uma seqüência
para i := 1,...,⌊n/2⌋ faça
        temp := S[i]
        S[i] := S[n − i + 1]
        S[n − i + 1] := temp
```

- Tamanho da entrada: n elementos da sequência
- Operação dominante: troca de elementos
  - Cada troca envolve três operações básicas
- Complexidade: n/2 operações dominantes

Complexidade de algoritmos

- Exemplo 2: Soma de matrizes

```
algoritmo 1.5: Soma de matrizes
para i := 1, ..., n faça
    para j := 1, ..., n faça
        c_ij := a_ij + b_ij
```

  - Tamanho da entrada?
  - Operação dominante?
  - Complexidade?

Complexidade de algoritmos

- Exemplo 3: Multiplicação de matrizes

```
algoritmo 1.6: Produto de matrizes
para i := 1,...,n faça
    para j := 1,...,n faça
        c_ij := 0
        para k := 1,...,n faça
            c_ij := c_ij + a_ik . b_kj
```

  - Tamanho da entrada?
  - Operação dominante?
  - Complexidade?

Complexidade de algoritmos

- Complexidade do pior caso: considera a situação onde o algoritmo executa o maior número de operações possível
  - Mais utilizada: prever recursos (CPU) para execução
- Complexidade do melhor caso: considera a situação onde o algoritmo executa o menor número de operações possível
- Complexidade do caso médio: considera todas as situações possíveis e suas probabilidades de ocorrência

- Complexidade de espaço: necessidade de memória, etc.

Complexidade de algoritmos

- Exemplo: Busca em vetor pelo elemento x
  - Operação dominante?
  - Complexidade do pior caso?
  - Complexidade do melhor caso?
  - Complexidade do caso médio?

Notação assintótica

- Análise assintótica
  - Estudo do comportamento de algoritmos com entradas grandes
  - Ordem de crescimento do tempo de execução em função do tamanho da entrada
- Em geral
  - Algoritmo assintoticamente mais eficiente: melhor
    - Exceto para entradas pequenas

Notação assintótica

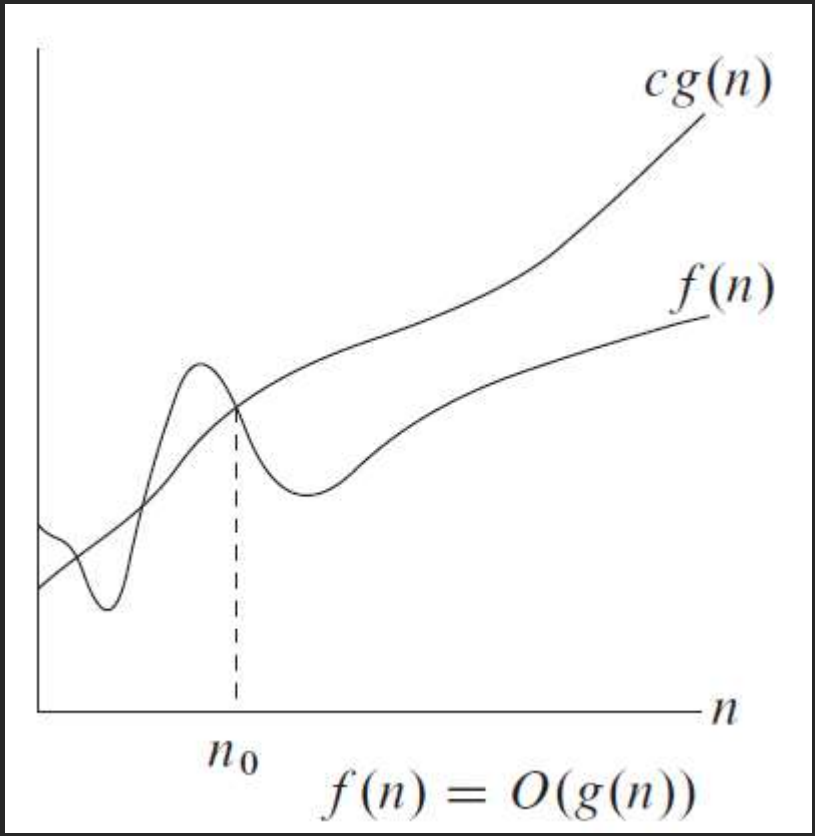
- Notações assintóticas
  - Notações utilizadas para descrever o tempo de execução assintótico de algoritmos
  - Exemplos:
    - O: limite superior
      - Em geral, pior caso
    - Ω: limite inferior
      - Em geral, melhor caso
    - Θ: limite superior e inferior justo
      - Ordem de crescimento

Notação O

- Dada função g(n)
  - O(g(n)) é um conjunto de funções
  - O(g(n)) = { f(n) : existem constantes positivas c e n_0 tais que

    $0 \leq f(n) \leq cg(n)$ para todo $n \geq n_0$ }
    - $f(n) \in O(g(n))$ se existem c e n_0 tais que f(n) nunca supere cg(n)
    - f(n) = O(g(n)) é sinônimo para $f(n) \in O(g(n))$
  - g(n) é limite assintótico superior para f(n)

Notação O



Notação O

- Exemplo: mostar que $f(n) = 20n^2 + 10n \in O(n^2)$
  - O que fazer?
    - Encontrar **constantes** positivas c e n_0 tais que $20n^2 + 10n \leq cn^2$ para todo $n \geq n_0$
- $f(n) = O(n^3)$?
- $f(n) = O(n^4)$?
- $f(n) = O(n \sqrt{n})$?
- $f(n) = O(n \lg n)$?

Notação Ω

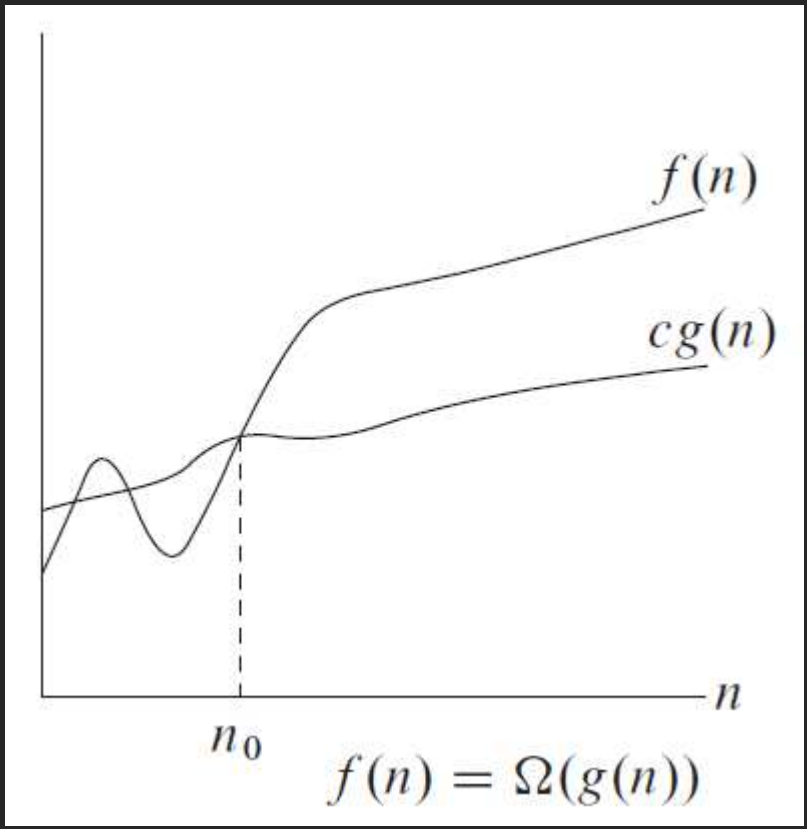
- Dada função g(n)
  - $\Omega(g(n))$ é um conjunto de funções
  - $\Omega(g(n)) = \{ f(n) :$ existem constantes positivas c e n_0 tais que

    $$0 \leq cg(n) \leq f(n) \text{ para todo } n \geq n_0 \}$$

    - $f(n) \in \Omega(g(n))$ se existem c e n_0 tais que f(n) nunca supere cg(n)
    - $f(n) = \Omega(g(n))$ é sinônimo para $f(n) \in \Omega(g(n))$

  - g(n) é limite assintótico inferior para f(n)

Notação Ω



Notação Ω

- Exemplo: mostar que $f(n) = \frac{n^2}{20} - 10n \in \Omega(n^2)$
  - O que fazer?
    - Encontrar **constantes** positivas c e n_0 tais que $cn^2 \leq \frac{n^2}{20} - 10n$ para todo $n \geq n_0$
- $f(n) = \Omega(n^3)$?
- $f(n) = \Omega(n^4)$?
- $f(n) = \Omega(n \sqrt{n})$?
- $f(n) = \Omega(n \lg n)$?

Notação Θ

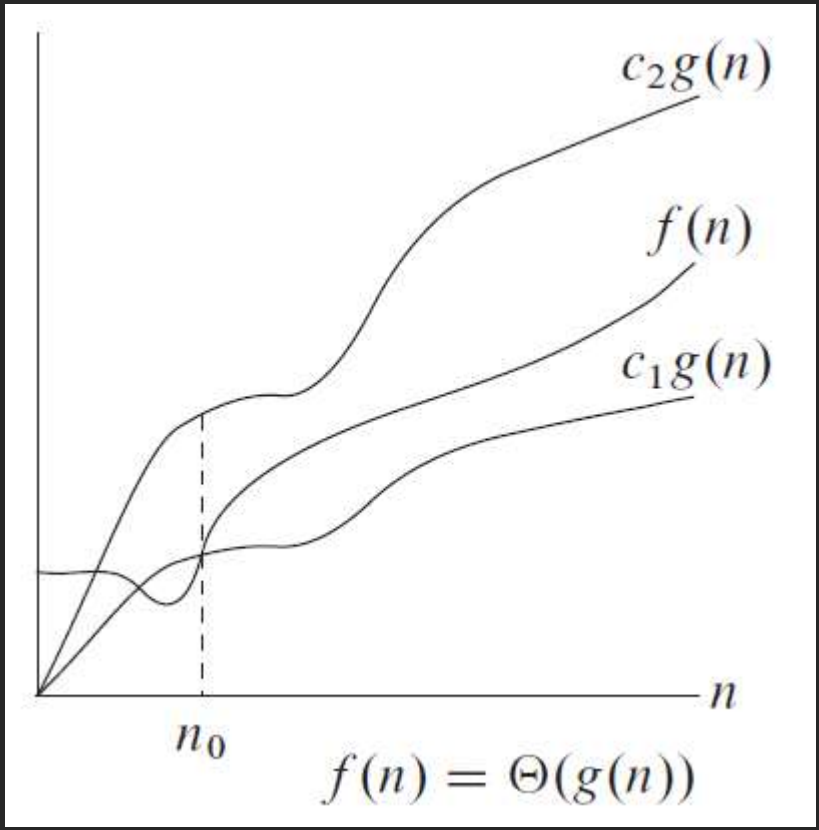
- Dada função g(n)
  - $\Theta(g(n))$ é um conjunto de funções
  - $\Theta(g(n)) = \{ f(n) :$ existem constantes positivas c_1, c_2 e n_0 tais que

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ para todo } n \geq n_0 \}$$

    - $f(n) \in \Theta(g(n))$ se existem c_1, c_2 e n_0 tais que f(n) difere de g(n) apenas por fatores constantes
    - $f(n) = \Theta(g(n))$ é sinônimo para $f(n) \in \Theta(g(n))$

  - g(n) é limite assintótico restrito (superior e inferior) para f(n)

Notação Θ



Notação Θ

- Exemplo: mostar que $f(n) = \frac{n^2}{2} - 3n \in \Theta(n^2)$
  - O que fazer?
    - Encontrar **constantes** positivas c_1, c_2 e n_0 tais que $c_1 n^2 \leq \frac{n^2}{2} - 3n \leq c_2 n^2$ para todo $n \geq n_0$
- $f(n) = \Theta(n^3)$?
- $f(n) = \Theta(n)$?

Algoritmo ótimo

- Seja um problema P que requer $\Omega(n)$ operações para ser resolvido para uma entrada de tamanho n.
- Algoritmo ótimo: algoritmo que resolve P com $O(n)$ operações
  - Um algoritmo ótimo apresenta a menor complexidade possível

Exercícios

- Livro Szwarcfiter:
  - 1.4, 1.14, 1.15, 1.17, 1.18, 1.19, 1.20